

BUILD YOUR FIRST GENERATIVE AI APPLICATION IN 60 MINUTES

AI with Durga Sir - Weekend Masterclass | Complete Printout-Friendly Teaching Script

Workshop Project: AI Learning Assistant

The user enters a topic and learner level. Our application generates a simple explanation, 5 important points, and one real-life example.

Main workshop goal: A complete beginner should understand how a simple Generative AI application is built step by step.

1. Opening

Friends, good evening everyone.

Welcome to AI with Durga Sir - Weekend Masterclass.

Today's topic is: BUILD YOUR FIRST GENERATIVE AI APPLICATION IN 60 MINUTES

Even if you are completely new to Generative AI, no problem.

Even if you don't know LangChain, no problem.

Today we are starting from zero.

By the end of this workshop, we are going to build one working Generative AI application using Python and LangChain.

Listen carefully.

There is a difference between **using ChatGPT** and **building our own application using an AI model**.

Today we are concentrating on the second one.

Write on the board:

```
USER
↓
OUR PYTHON APPLICATION
↓
AI MODEL
↓
ANSWER
```

Our final application will look like this:

```
USER INPUT
↓
PromptTemplate
↓
Chat Model
↓
StrOutputParser
↓
FINAL ANSWER
```

Very Very Important: Today's application is a Generative AI application, not an Agentic AI application.

Here, the model mainly receives our instruction and generates content.

2. Basic Generative AI Application Architecture

Friends, before writing Python, understand only three words:

- **Prompt**
- **Model**

- **Output**

Write on the board:

PROMPT → MODEL → OUTPUT

What is a Prompt?

A prompt is the input or instruction that we give to an AI model.

Example:

Explain Python in simple English.

That is a prompt.

Another example:

Give me 5 Java interview questions.

That is also a prompt.

PROMPT = INPUT / INSTRUCTION GIVEN TO THE MODEL

Now, what is a Model?

The model is the intelligence that processes our input and generates a response.

What is Output?

Output is the result generated by the model.

Prompt = Input

Model = Processing

Output = Result

Think about a restaurant. You say, "One masala dosa." That is your request. The kitchen processes the request. Finally, you receive dosa.

ORDER → KITCHEN → FOOD

PROMPT → MODEL → OUTPUT

Master Point: A simple Generative AI application starts with Prompt → Model → Output.

Have you got my point?

3. First Generative AI Program

Now enough theory. Let us write our first Generative AI program. Initially, do not use PromptTemplate. Do not use Output Parser. Do not use LCEL. Keep it very simple.

```
import os

from langchain_openai import ChatOpenAI

model = ChatOpenAI(
    model="gpt-4o-mini",
    api_key=os.getenv("OPENAI_API_KEY")
)

response = model.invoke(
    "Explain Generative AI in very simple English."
)

print(response.content)
```

Now explain the program.

First:

```
model = ChatOpenAI(...)
```

We created our Chat Model object.

Next:

```
response = model.invoke(  
    "Explain Generative AI in very simple English."  
)
```

invoke() means: give one input to the model and get one response.

Finally:

```
print(response.content)
```

We are printing the generated answer.

Our flow is:

```
"Explain Generative AI..."  
    ↓  
Chat Model  
    ↓  
AIMessage  
    ↓  
.content  
    ↓  
Answer
```

4. What Exactly Is response?

Friends, one important question.

Is response simply a normal Python string?

No.

Let Python prove it.

```
print(type(response))
```

Now run:

```
import os

from langchain_openai import ChatOpenAI

model = ChatOpenAI(
    model="gpt-4o-mini",
    api_key=os.getenv("OPENAI_API_KEY")
)

response = model.invoke(
    "Explain Generative AI in one sentence."
)

print(type(response))

print(response)

print(response.content)
```

Explain:

response



Complete AIMessage Object

response.content



Generated Answer Content

Very Very Important: A Chat Model normally returns an AIMessage, not just a plain Python string.

For this workshop, do not go deeply into AIMessage. Just remember:

Chat Model



AIMessage



.content



Answer

Learning Conclusion: response is the complete message object; response.content gives the main answer text.

5. Problem With Our First Application

Friends, our application is working. But one problem is there.

Observe:

"Explain Generative AI in very simple English."

What is fixed here?

Generative AI.

Suppose tomorrow I want Python. I need to change the program. Java? Again change. AWS? Again change. Machine Learning? Again change. Ayyooo!

This is not a reusable application.

STATIC PROMPT

Explain Generative AI in simple English.

But we need:

DYNAMIC PROMPT

Explain {topic} in simple English.

Here {topic} can change.

topic = Python

topic = Java

topic = Generative AI

Very Important: Static prompt means fixed input. Dynamic prompt means some values can change.

This brings us to PromptTemplate.

6. What Is PromptTemplate?

PromptTemplate is used to create reusable text prompts with variables.

Think like this:

TEMPLATE

+

VALUES

↓

FINAL PROMPT

Today we are creating: AI LEARNING ASSISTANT

The user will provide:

Topic

Learner Level

For example:

Topic = Python

Level = Beginner

Create our PromptTemplate:

```
from langchain_core.prompts import PromptTemplate
```

```
prompt = PromptTemplate.from_template(
```

```
    """
```

```
    Explain {topic} to a {level} learner.
```

```
    Requirements:
```

- Use very simple English
- Give a simple definition
- Give 5 important points
- Give one real-life example

```
    """
```

```
)
```

Observe carefully. We have:

```
{topic}  
{level}
```

These are input variables.

Very Important: PromptTemplate gives us a reusable prompt structure.

7. Invoke PromptTemplate

Now let us provide values.

```
prompt_value = prompt.invoke(  
    {  
        "topic": "Generative AI",  
        "level": "beginner"  
    }  
)  
  
print(prompt_value.to_string())
```

Ask students: Did OpenAI execute here?

No.

Very very important. We only invoked `prompt.invoke(...)`. That means we invoked `PromptTemplate`.

It prepared the final prompt. It did not generate the AI answer.

```
prompt.invoke()
```



PREPARES PROMPT

```
model.invoke()
```



CALLS AI MODEL



GENERATES RESPONSE

Very Very Important: `prompt.invoke()` and `model.invoke()` are not the same.

`prompt.invoke()` prepares the input.

`model.invoke()` sends input to the model and generates a response.

8. Connect PromptTemplate With Model

Now we have two components: PromptTemplate and Chat Model. Let us connect them manually.

```
import os
```

```
from langchain_openai import ChatOpenAI
```

```
from langchain_core.prompts import PromptTemplate
```

```
model = ChatOpenAI(  
    model="gpt-4o-mini",  
    api_key=os.getenv("OPENAI_API_KEY")  
)
```

```

prompt = PromptTemplate.from_template(
    """
    Explain {topic} to a {level} learner.

    Requirements:
    - Use very simple English
    - Give a simple definition
    - Give 5 important points
    - Give one real-life example
    """
)

prompt_value = prompt.invoke(
    {
        "topic": "Generative AI",
        "level": "beginner"
    }
)

response = model.invoke(
    prompt_value
)

print(response.content)

```

Understand the flow:

Input Values



PromptTemplate



Formatted Prompt



Chat Model



AIMessage



Answer

Master Statement: Output of PromptTemplate becomes input to the Chat Model.

Remember this carefully. This same idea will become the foundation for LCEL.

9. Take Input From the User

Until now we again hardcoded Generative AI and beginner. Let us make the application interactive.

```
topic = input("Enter topic: ")
```

```
level = input("Enter learner level: ")
```

Then:

```
prompt_value = prompt.invoke(  
    {  
        "topic": topic,  
        "level": level  
    }  
)  
  
response = model.invoke(  
    prompt_value  
)  
  
print(response.content)
```

Run with:

```
Enter topic: Python  
Enter learner level: Beginner
```

Again run:

```
Enter topic: Machine Learning  
Enter learner level: School Student
```

Ask: Did we change our Python program?

No. Only the input values changed.

Very Important: A reusable application should accept changing input without requiring us to rewrite the program.

10. What Is Output Processing?

Friends, our model gives an AIMessage.

Suppose our Python application needs a plain string. We already know response.content. That is perfectly okay.

But LangChain also provides output-processing components. One simple component is StrOutputParser.

```
AIMessage
  ↓
StrOutputParser
  ↓
String
```

StrOutputParser processes model output and gives us plain string output.

Import:

```
from langchain_core.output_parsers import StrOutputParser
```

Create:

```
parser = StrOutputParser()
```

Use:

```
result = parser.invoke(
    response
)

print(result)
```

Now check:

```
print(type(response))

print(type(result))
```

Conceptually:

Before Parser



AIMessage

After Parser



str

**MODEL GENERATES THE ANSWER.
PARSER PROCESSES THE ANSWER.
MODEL ≠ PARSER**

11. Complete Manual Generative AI Application

Now combine everything.

```
import os

from langchain_openai import ChatOpenAI
from langchain_core.prompts import PromptTemplate
from langchain_core.output_parsers import StrOutputParser

# =====
# CREATE CHAT MODEL
# =====

model = ChatOpenAI(
    model="gpt-4o-mini",
    api_key=os.getenv("OPENAI_API_KEY")
)
```



```

# =====
# CREATE PROMPT TEMPLATE
# =====

prompt = PromptTemplate.from_template(
    """
    Explain {topic} to a {level} learner.

    Requirements:
    - Use very simple English
    - Give a simple definition
    - Give 5 important points
    - Give one real-life example
    """
)

# =====
# CREATE OUTPUT PARSER
# =====

parser = StrOutputParser()

# =====
# TAKE USER INPUT
# =====

```

```

topic = input("Enter topic: ")

level = input("Enter learner level: ")

# =====
# PREPARE PROMPT
# =====

prompt_value = prompt.invoke(
    {
        "topic": topic,
        "level": level
    }
)

# =====
# CALL AI MODEL
# =====

response = model.invoke(
    prompt_value
)

# =====
# PROCESS OUTPUT
# =====

```

```

result = parser.invoke(
    response
)

# =====
# DISPLAY RESULT
# =====

print("\nAI RESPONSE")
print("-----")

print(result)

```

Now write the complete architecture:

```

topic + level
  ↓
PromptTemplate
  ↓
Formatted Prompt
  ↓
Chat Model
  ↓
AIMessage
  ↓
StrOutputParser
  ↓
String

```

Master Conclusion: PromptTemplate prepares the input.

Master Conclusion: Chat Model generates the response.

Master Conclusion: StrOutputParser processes the output.

12. Introduce LCEL

Friends, our application is already working correctly. But observe carefully.

We manually wrote separate invoke calls for prompt, model, and parser. We manually connected the three components.

LangChain gives us a beautiful way to connect these components.

It is called LCEL - LangChain Expression Language.

Do not get afraid by the word "Language". We are still writing Python.

Write this in big letters:

```
chain = prompt | model | parser
```

THIS IS TODAY'S HERO LINE

13. Understand the Pipe Operator

A | B

Simple meaning:

Output of A

↓

Input to B

Therefore:

A | B | C

means

INPUT

↓

A

↓

B

↓

C

↓

OUTPUT

Now apply it to our application:

PromptTemplate

↓

Chat Model

↓

Parser

LCEL representation:

prompt | model | parser

Very Very Important: In an LCEL pipeline, the output of the left component becomes the input to the right component.

Prompt output



Model input

Model output



Parser input

Simple logic friends.

14. Manual Approach vs LCEL

Manual approach:

```
prompt_value = prompt.invoke(data)

response = model.invoke(prompt_value)

result = parser.invoke(response)
```

LCEL approach:

```
chain = prompt | model | parser

result = chain.invoke(data)
```

Same workflow. Different style.

The manual approach helps us understand every step. LCEL gives us a cleaner way to compose those steps.

Very Important Learning Sequence: First understand the components. Then understand the data flow. Then connect the components using LCEL.

15. Final Generative AI Application

Now create our final workshop program.

```
import os

from langchain_openai import ChatOpenAI
from langchain_core.prompts import PromptTemplate
from langchain_core.output_parsers import StrOutputParser

# =====
# CREATE MODEL
# =====

model = ChatOpenAI(
    model="gpt-4o-mini",
    api_key=os.getenv("OPENAI_API_KEY")
)

# =====
# CREATE PROMPT
# =====

prompt = PromptTemplate.from_template(
    """
    Explain {topic} to a {level} learner.

    Requirements:
    - Use very simple English
    """
)
```

- Give a simple definition
- Give 5 important points
- Give one real-life example

"""

)

=====

CREATE PARSER

=====

parser = StrOutputParser()

=====

CREATE GENERATIVE AI PIPELINE

=====

chain = prompt | model | parser

=====

TAKE USER INPUT

=====

topic = input("Enter topic: ")

level = input("Enter learner level: ")

=====


```

# RUN APPLICATION
# =====

result = chain.invoke(
    {
        "topic": topic,
        "level": level
    }
)

# =====
# DISPLAY RESULT
# =====

print("\nAI RESPONSE")
print("-----")

print(result)

```

Friends, congratulations. We have successfully built our first Generative AI application.

16. Final Architecture

Write this on the board:

USER
↓
Topic + Level
↓
PromptTemplate
↓
Chat Model
↓
StrOutputParser
↓
FINAL ANSWER

And finally:

```
chain = prompt | model | parser
```

Most Important Formula: PromptTemplate → prepares input

Most Important Formula: Chat Model → generates answer

Most Important Formula: StrOutputParser → processes output

Most Important Formula: LCEL → connects the components

17. Quick Live Experiment

Ask somebody from the audience: Give me one topic.

Topic: Java
Level: Beginner

Run it. Then try:

Topic: Generative AI
Level: School Student

Again:

Topic: AWS

Level: Interview Candidate

Ask: Did we change our Python application?

No. Same application. Different input. Different result.

Learning Conclusion: A reusable Generative AI application can work with different inputs without changing the complete program.

18. Generative AI vs Agentic AI

Before closing, clarify one point. Ask: Is today's application Generative AI or Agentic AI?

GENERATIVE AI

Prompt

↓

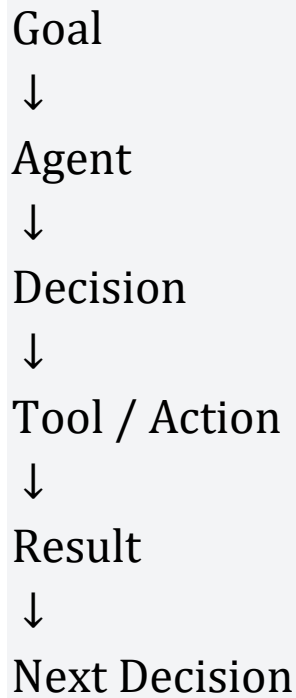
Model

↓

Generated Response

The model's main job here is to generate content.

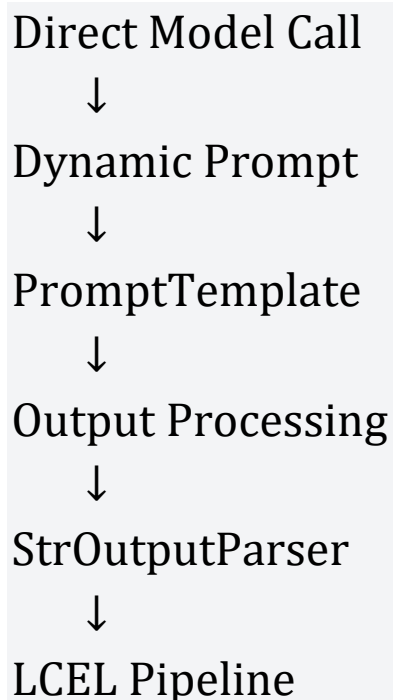
Later, Agentic AI may involve:



Today's workshop gives us the foundation. Agentic AI comes later when the application starts making decisions and taking actions.

19. Final Revision

Friends, today our journey was:



Our complete architecture:

```
INPUT
↓
PROMPT
↓
MODEL
↓
OUTPUT PROCESSING
↓
RESULT
```

And the most important Python line:

```
chain = prompt | model | parser
```

Five Points Students Must Remember

- **A prompt is the input or instruction given to the AI model.**
- **PromptTemplate creates reusable dynamic prompts.**
- **A Chat Model generates the AI response and normally returns an AIMessage.**
- **StrOutputParser processes the model output and gives us a string.**
- **LCEL connects compatible components into a pipeline.**

20. Closing Script

Friends, today we started from zero.

```
Prompt → Model → Output
```

Then we wrote our first direct model program.

Then we understood: Chat Model → AIMessage → .content

Then we identified the problem with hardcoded prompts.

We introduced PromptTemplate.

Then we made our application dynamic.

Next, we introduced StrOutputParser.

Finally, we connected everything using LCEL:

```
chain = prompt | model | parser
```

So today you did not simply use an AI tool. You understood how the building blocks of a Generative AI application work together.

From here, our learning journey can continue towards Structured Output, Conversation Applications, RAG, Tools, Agents, Agentic AI and LangGraph.

But for today's workshop, remember only this:

```
INPUT
↓
PROMPT
↓
MODEL
↓
OUTPUT
```

And remember today's hero line:

```
chain = prompt | model | parser
```

Final Learning Conclusion: First understand the components. Then understand the data flow. Finally connect the components.

Have you got my point?

Thank you everyone.